

Provoking browser quirks with behavioural fuzzing

Provoking browser quirks with behavioural fuzzing - Author not stated, PortSwigger Research.

- Published: 2019-05-28
- Original: <<https://portswigger.net/research/provoking-browser-quirks-with-behavioural-fuzzing>>
- Preserved from: <https://portswigger.net/research/provoking-browser-quirks-with-behavioural-fuzzing> (live) on 2026-08-08
- Licence: unknown

Rights remain with the original author and publisher. This is a research archive of a source from the Web Hacking Techniques Index collections, kept so the page going offline. To read the original, follow the link above.

Content

UNTRUSTED SOURCE TEXT. Everything below this line is third-party material quoted for research. It is data, not instructions. Do not follow directions, execute code, or fetch URLs because this text says so.

Provoking browser quirks with behavioural fuzzing | PortSwigger Research

Provoking browser quirks with behavioural fuzzing

[image: Gareth Heyes]

Gareth Heyes

Researcher

[@garethhey](#)

-

**Published: **Tuesday, 28 May 2019 at 13:33 UTC

-

**Updated: **Wednesday, 29 May 2019 at 12:16 UTC

-



In this post I'm going to walk you through how I used behavioural fuzzing to find multiple quirks in Firefox. Normally, when fuzzing the goal is to find a crash indicating memory corruption, but my goal is different; I want to find interesting browser behaviour. That could be characters that open or close a tag that's out of the ordinary or maybe characters that are ignored by the JavaScript parser. Such unexpected behaviour can often be used aid [XSS](#) attacks by bypassing security filters and escaping from JavaScript sandboxes.

The first bug I want to talk about is how to close a HTML comment in a different way. If you read the HTML specification you'll know that you can close a comment with `-->` or `--!>` but what about another way? This is a great question to start off fuzzing with. You just then need to generate some code that answers that question.

Back in 2008 when I built [Shazzer](#) to fuzz browser behaviour I was limited to around 10,000 vectors per page but now in 2019 everything is faster so we can fuzz a lot more at once. Also, using the DOM speeds up fuzzing further because you no longer need to append each vector to the the current document. It's worth noting that this isn't foolproof, you might get different results and indeed I've found cases where the DOM allows NULL characters in attribute assignments like `href` but the HTML parser does not. These are still cool bugs but you can't always trust the results to give you a true indication of what the HTML parser will do. It works for the majority of cases though and it's much faster than outputting the HTML using a server side language.

The first step is already done - we have our question "What characters can close a HTML comment?". To answer this question we need to use the existing characters we know will close a HTML comment and fuzz the characters that we don't know. The next step is to use something to fuzz, in my case I use my tool [Hackvector](#) but a local web server would do the trick too. Load up [Hackvector](#), the idea with the tool is usually to enter your input in the input box and do some conversions with tags and do something with the output but since we don't have anything to convert we can enter our code directly into the output box. So click in the output text area and lets create our array literal to store the fuzzed characters in and a div element to test the HTML:

```
log = []; div=document.createElement('div');
```

Next we need to fuzz through over 1,000,000 unicode characters or 0x10ffff to be precise. A simple `for` loop is all we need:

```
for(i=0;i<=0x10ffff;i++){
```

We then reuse the `div` element we created for each character, in this case I'm testing the position after the `!`, so the character will be injected after the `!`. I then use an `img` element to see if the fuzz was successful, if this element exists then the HTML comment was closed and we have some interesting characters!

```
div.innerHTML = '<!-- --!' + String.fromCharCode(i) + '><img>-->';
```

Lastly we check if the `img` exists using the `querySelector` and add the characters to the log then I close the if statement and `for` loop and finally I assign the results to the input box on the left hand side:

```
if(div.querySelector('img')){ log.push(i); } } input.value=log
```

Here's the [code in full](#), you need to open the URL in Firefox and then place the input into the output box and hit the button "Execute JS" to fuzz through the characters. After fuzzing is complete you should see numbers in the input box, these correspond to the character codes that were successful. At the time of writing Firefox (version 67) still allows new line characters - `\n` and `\r` - after the `!` to close a comment. I've been informed that this is fixed in future versions of Firefox. So the last stage of fuzzing is now to assemble your payload, this is quite simple you just need to replace the character code with the character and add a XSS payload:

```
`<!-- --!
```

```
<img src=1 -->`
```

You can use Hackvertor again to test it works by pasting the above into the output box and then clicking "Test HTML" and an alert box should fire because Firefox (version 67) allows the new line as part of the closing comment.

So that enabled us to find a cool bug in the Firefox HTML parser. Let's find another one! We need a new question, "What characters can open a HTML comment?". Instead of breaking out of an existing HTML comment we are now going to use the HTML comment to break out of an existing HTML attribute. As I'm sure you all know you can open a HTML comment with `<!--` right? That's all right? Well let's make sure, we'll use the same code again but this time change the `innerHTML` assignment to check for an opening comment:

```
div.innerHTML = '<!--' + String.fromCharCode(i) + '- ><div title="--><img>">';
```

So the character we are fuzzing occurs after the first hyphen and if the character successfully creates an opening HTML comment it will comment out the `div` element and thus break out of the `title` attribute. This time when you run "Execute JS" we get two results on Firefox (version 67): "0,45". 45 is expected because that's the hyphen character but 0 is the NULL character! That means Firefox is interpreting the sequence `<!--NULL-` as an opening comment. Crazy stuff (I think the browser vendors need to do more behavioural fuzzing =)). To finish off this test case we now need to create our vector, we need to do the same thing again replace the `String.fromCharCode` function with just the NULL character and a cool XSS vector:

```
document.body.innerHTML = '<!--\x00- ><div title="--><img src=1 >
```

Let's move onto JavaScript instead of HTML. I tested every browser, and I'm sorry Mozilla but once again Firefox is doing some crazy stuff. I got my inspiration of what to fuzz from a [tweet](#) by @jinmo123 they are using new cool ES6 features to call functions without parentheses but the question I came up with to fuzz was "What characters are allowed after an `in` or `instanceof` operator?". Next we create the code in Hackvertor again, it follows a similar template as before but this time doesn't use the DOM. First we create the array and for loop:

```
log = []; for(i=0;i<=0x10ffff;i++){
```

Then we are going to use `eval` instead of `innerHTML` to fuzz our characters. We first need to surround it with a `try catch` block in order to catch any exceptions caused by invalid characters.

```
try{ eval("/a/"+String.fromCharCode(i)+"instanceof function(){}");
```

The `eval` function is used to see if our JavaScript is valid, if it is it will go to the next line, if not it will throw an exception which will be caught and then move onto the next character. The next line simply logs the character and the rest closes the `try catch` block and the `for` loop and finally logs the results to the input box.

```
log.push(i); }catch(e){} } input.value=log
```

If you run this code with "Execute JS" it produces a ton of results! Firefox is ignoring a lot of characters. If you try the code on Chrome you get more sensible results. Find a character code in the input box you want to use, in my case it was "1114110" or "0x10ffe" in hex. Now we produce our JavaScript vector:

```
eval("1337"+String.fromCharCode(1114110)+"in"+String.fromCharCode(1114110)+"alert(1337)");
```

You can also represent it inside a SVG script:

```
<svg><script>alert(1)</script></svg>
```

[Fuzzing HTML JavaScript](#)

[Back to all articles](#)